

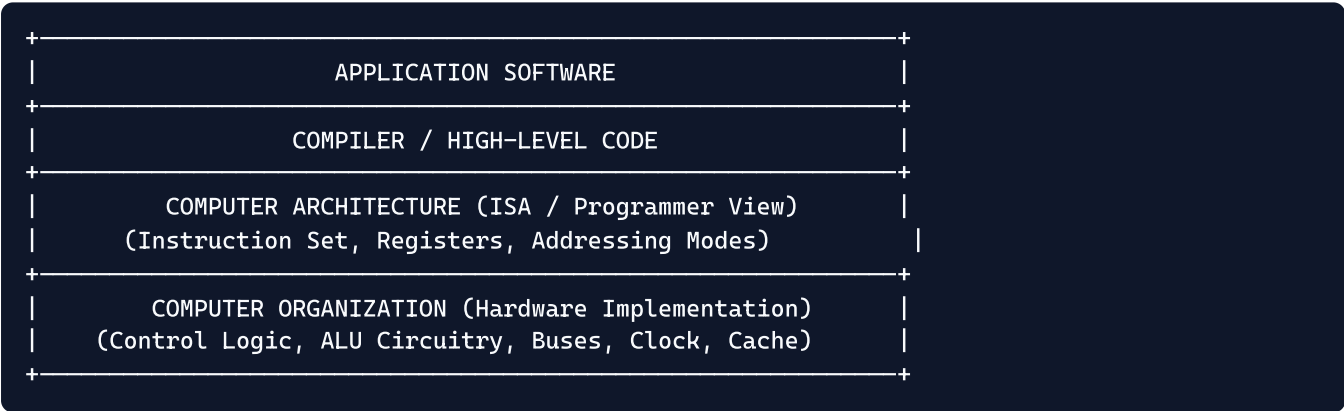
MPCA Module 1 — 10 Mark Questions & Answers

Q1. Explain Computer Architecture vs Computer Organization in detail with a suitable diagram and real-world example (e.g., same architecture, different organization).

1. Fundamental Distinction

- **Computer Architecture:** Refers to those attributes of a system visible to a programmer, which directly impact the logical execution of a program. It defines the abstract software-hardware interface (ISA — Instruction Set Architecture).
- *Key Attributes:* Instruction set design, supported data types, register set layout, addressing modes, and memory byte ordering.
- **Computer Organization:** Refers to the operational units and their physical interconnections that realize the architectural specifications.
- *Key Attributes:* Clock frequency, control signal timing, cache technology, memory bus width, and physical gate design.

2. Conceptual Diagram



3. Comprehensive Comparison Table

Feature	Computer Architecture	Computer Organization
Perspective	High-level programmer / compiler view.	Low-level hardware designer / engineer view.
Function	Specifies <i>what</i> the processor can do.	Specifies <i>how</i> the processor performs it.
Stability	Long-term stability (architectures persist decades).	Frequently updated with technology advances.
Design Factors	Operands, instruction formats, logic operations.	Clock speed, bus width, pipelining, cache sizes.
Example	x86-64 ISA, ARMv8 ISA.	Intel Core i7 vs Intel Core i9 implementations.

4. Real-World Example: Same Architecture, Different Organization

The **x86 Architecture** (originally introduced with Intel 8086) has remained architecturally compatible for decades:

* An **Intel Core i3** and an **Intel Core i9** processor share the **same Computer Architecture** (x86-64 ISA). Both run the exact same compiled binary program and support identical assembly instructions (e.g., `MOV`, `ADD`, `PUSH`).

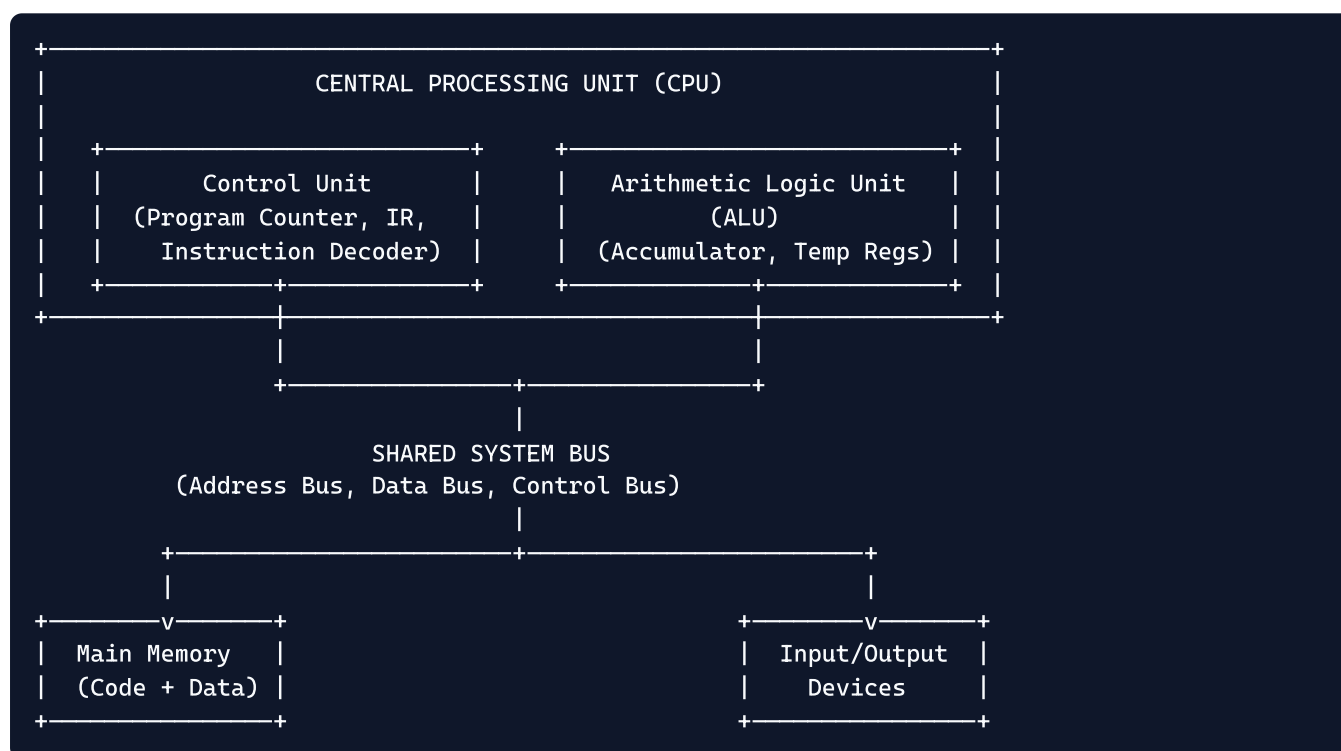
* However, they have radically **different Computer Organizations**:

* **Core i3**: 4 physical cores, 6 MB L3 cache, lower clock rate, basic pipeline.

* **Core i9**: 24 physical cores, 36 MB L3 cache, higher clock rate, advanced out-of-order execution engine, superscalar pipelines.

Q2. Describe the Von Neumann model in detail — architecture diagram, components, key features, instruction cycle (Fetch-Decode-Execute-Store), advantages and disadvantages.

1. Architecture Diagram



2. Major Components

1. **Control Unit (CU)**: The central manager. Fetches instructions, decodes opcodes, and issues timing/control signals to all internal components.
2. **Arithmetic Logic Unit (ALU)**: Performs integer arithmetic (+, -, ×) and logical operations (AND, OR, NOT, shifts).
3. **Registers**: Internal high-speed storage:
 - * **Program Counter (PC)**: Holds memory address of the next instruction.
 - * **Instruction Register (IR)**: Stores current instruction opcode being decoded.
 - * **Accumulator (AC)**: Holds intermediate arithmetic results.
4. **Memory Unit (RAM)**: Stores both program instructions and data values in unified binary storage.

5. **System Bus:** Single shared physical bus comprising Address Bus, Data Bus, and Control Bus.

3. Key Features

- **Stored Program Concept:** Code and data share a single memory space and bus.
- **Sequential Execution:** Instructions are fetched and executed one after another unless a branch instruction alters the Program Counter.
- **Unified Memory Addressing:** Memory locations are linearly addressed regardless of whether they store data or code.

4. Instruction Cycle Flow

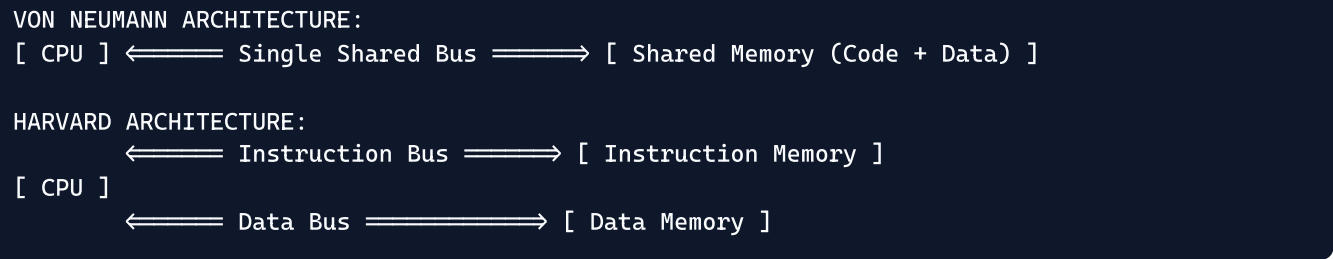
Fetch Instruction (from memory at PC) → Increment PC → Decode Instruction in IR
→ Fetch Operands (if needed) → Execute Operation in ALU → Store Result (Write-back)

5. Advantages and Disadvantages

- **Advantages:**
 - Hardware simplicity and low manufacturing cost due to single bus architecture.
 - Maximum memory utilization (flexible code vs data allocation).
 - Easy loading and swapping of dynamic software.
- **Disadvantages:**
 - **Von Neumann Bottleneck:** CPU speed is limited because instruction fetch and data access cannot occur simultaneously over the shared bus.
 - **Security Vulnerabilities:** Code and data reside in the same memory, enabling buffer overflow attacks where data overwrites code.

Q3. Compare Von Neumann and Harvard architecture in detail with diagrams and a comparison table, and discuss which is better for modern processors and why.

1. Architectural Diagrams



2. Detailed Comparison Table

Feature	Von Neumann Architecture	Harvard Architecture
Memory Structure	Single unified memory space for code and data.	Separate physical memory blocks for code and data.
Bus Layout	Single set of Address, Data, and Control buses.	Separate Instruction Bus and Data Bus.
Simultaneous Access	Cannot fetch code and read/write data concurrently.	Can fetch instruction and access data in the same clock cycle.
Execution Speed	Slower (bottlenecked by shared bus).	Faster (supports simultaneous bus activity).
Hardware Complexity	Simple design; low pin count and board complexity.	Complex design; higher pin count and separate bus controllers.
Memory Efficiency	High (unused code space can store data).	Strict separation (unused code memory is wasted for data).
Primary Usage	General Purpose Computers (PCs, Servers).	Embedded Microcontrollers, DSPs, CPU Caches.

3. Which is Better for Modern Processors and Why?

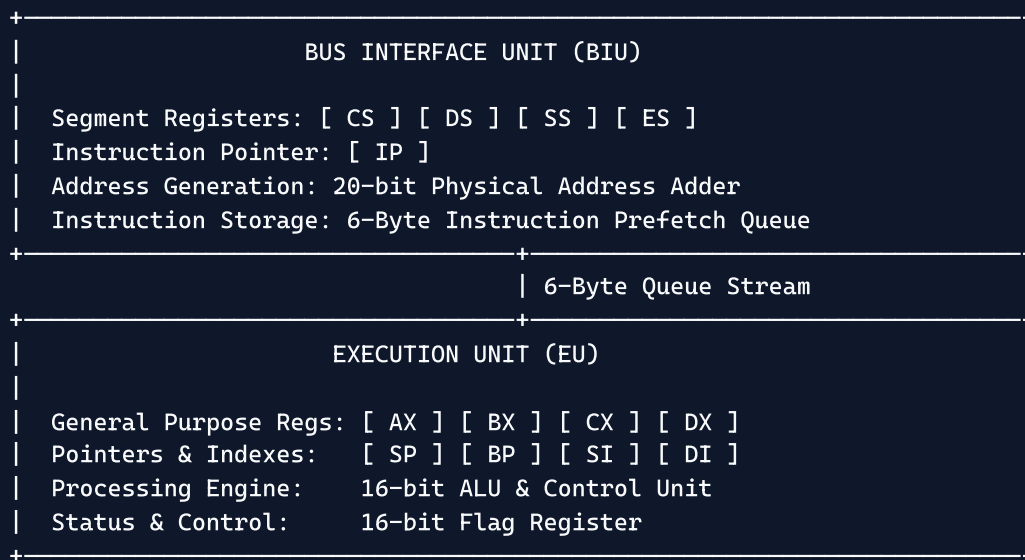
Modern high-performance processors (like Intel Core, AMD Ryzen, ARM Cortex-A) use a **Modified Harvard Architecture** internally, combined with a **Von Neumann Architecture** externally:

- **Internal Level (Harvard Concept for Speed):**
- Processors use separate **L1 Instruction Cache** and **L1 Data Cache**.
- This allows the instruction fetch unit and data load/store unit to operate concurrently at full clock speed without bus contention, enabling deep instruction pipelining.
- **External Level (Von Neumann Concept for Flexibility):**
- Outside the CPU chip, main RAM is unified into a single memory block.
- This allows operating systems to dynamically allocate memory space to programs and data without wasting physical memory chips.

Conclusion: The **Modified Harvard Architecture** is superior for modern high-speed processors because it delivers the ultra-fast execution of dual-bus Harvard caching inside the CPU while preserving the cheap, flexible unified RAM of Von Neumann on the motherboard.

Q4. Describe the architecture of the 8086 microprocessor in detail with a block diagram — explain BIU, EU, registers, operating modes, and memory organization.

1. Internal Block Diagram



2. Functional Units: BIU vs EU

- **Bus Interface Unit (BIU):**

- Manages physical bus transfers (fetching instructions, memory read/write, I/O access).
- Contains segment registers (CS, DS, SS, ES) and Instruction Pointer (IP).
- Calculates 20-bit physical addresses: $\text{Physical Address} = (\text{Segment} \times 16) + \text{Offset}$.
- Houses a **6-byte Instruction Prefetch Queue** that fetches instructions asynchronously while the EU is executing current code.

- **Execution Unit (EU):**

- Decodes and executes instructions fetched from the BIU queue.
- Contains the 16-bit ALU, general-purpose registers, pointer/index registers, and Flag register.
- Operates completely isolated from external memory buses.

3. Register Organization

1. General-Purpose Registers (16-bit / 8-bit pairs):

- * **AX (AH, AL):** Accumulator (used in multiplication, division, I/O).
- * **BX (BH, BL):** Base Register (holds base offset for memory addressing).
- * **CX (CH, CL):** Count Register (used as loop counter and shift count).
- * **DX (DH, DL):** Data Register (holds implicit operands, I/O port addresses).

2. Pointer and Index Registers (16-bit):

- * **SP (Stack Pointer):** Offset of current top of stack.
- * **BP (Base Pointer):** Base offset for accessing stack parameters.
- * **SI (Source Index):** Source offset in string operations.
- * **DI (Destination Index):** Destination offset in string operations.

3. Segment Registers (16-bit):

- * **CS** (Code Segment), **DS** (Data Segment), **SS** (Stack Segment), **ES** (Extra Segment).

4. Memory Organization (Segmentation)

- 20-bit address bus gives **1 MB (2^{20} bytes)** total memory space.
- Memory is divided into logical **64 KB segments**.
- Allows 16-bit registers to address 1 MB space via segment:offset pairing.

5. Operating Modes

- **Minimum Mode (MN/MX' = 5V):** Single-processor mode; 8086 generates all control signals directly.
- **Maximum Mode (MN/MX' = 0V):** Multi-processor mode; control signals are generated by an external **8288 Bus Controller**.

Q5. Explain all the performance measures of computer architecture with formulas, and solve a numerical problem using them.

1. Performance Measures & Formulas

1. **CPU Execution Time (T_{cpu}):** Total time taken to execute a program.

$$T_{\text{cpu}} = \text{Instruction Count (IC)} \times \text{CPI} \times \text{Clock Cycle Time } (T_c)$$

$$T_{\text{cpu}} = \frac{\text{IC} \times \text{CPI}}{\text{Clock Rate } (f)}$$

2. **Performance (P):** Inverse of execution time.

$$P = \frac{1}{T_{\text{cpu}}}$$

3. **Clock Rate (f):** Clock frequency of the processor oscillator.

$$f = \frac{1}{T_c}$$

4. **CPI (Cycles Per Instruction):** Average clock cycles required per instruction.

$$\text{CPI} = \frac{\text{Total CPU Clock Cycles}}{\text{Instruction Count}}$$

5. **MIPS (Millions of Instructions Per Second):**

$$\text{MIPS} = \frac{\text{Instruction Count}}{T_{\text{cpu}}} \times 10^6 = \frac{f}{\text{CPI} \times 10^6}$$

6. **MFLOPS (Millions of Floating-Point Operations Per Second):**

$$\text{MFLOPS} = \frac{\text{Floating Point Operations}}{T_{\text{cpu}}} \times 10^6$$

7. **Throughput:** Number of complete execution tasks finished per unit time.

8. **Latency:** Time delay between task submission and completion.

2. Numerical Problem & Solution

Problem: A 3.0 GHz processor executes a benchmark program with 20 million instructions. The instruction mix and CPI breakdown are as follows:

- * Arithmetic Instructions: 50% of total instructions, CPI = 2
- * Data Transfer Instructions: 30% of total instructions, CPI = 3
- * Control Branch Instructions: 20% of total instructions, CPI = 4

Calculate:

1. Average CPI of the program
2. Total CPU Execution Time
3. MIPS Rating

Solution:

- **Step 1: Calculate Average CPI**

$$\text{Average CPI} = \sum (\text{Percentage}_i \times \text{CPI}_i)$$

$$\text{Average CPI} = (0.50 \times 2) + (0.30 \times 3) + (0.20 \times 4)$$

$$\text{Average CPI} = 1.0 + 0.9 + 0.8 = 2.7$$

- **Step 2: Calculate CPU Execution Time**

- $\text{Instruction Count (IC)} = 20 \times 10^6$

- $\text{Clock Rate } (f) = 3.0 \text{ GHz} = 3.0 \times 10^9 \text{ Hz}$

$$T_{\text{cpu}} = \frac{\text{IC} \times \text{CPI}}{f} = \frac{(20 \times 10^6) \times 2.7}{3.0 \times 10^9}$$

$$T_{\text{cpu}} = \frac{54 \times 10^6}{3.0 \times 10^9} = 18 \times 10^{-3} \text{ seconds} = 18 \text{ ms}$$

- **Step 3: Calculate MIPS Rating**

$$\text{MIPS} = \frac{f}{\text{CPI} \times 10^6} = \frac{3.0 \times 10^9}{2.7 \times 10^6} = 1111.11 \text{ MIPS}$$

Q6. Explain the complete instruction set of 8086 — arithmetic, data transfer, control transfer, logical, and shift/rotate instructions — with syntax and examples for each category.

1. Data Transfer Instructions

Used to transfer binary data between registers, memory, and I/O ports.

- * `MOV dest, src` $\rightarrow$ `MOV AX, BX` (Copies BX to AX).
- * `PUSH src` $\rightarrow$ `PUSH CX` (Decrements SP by 2, stores CX on stack).
- * `POP dest` $\rightarrow$ `POP DX` (Reads stack into DX, increments SP by 2).
- * `LEA dest, src` $\rightarrow$ `LEA SI, [BX+1000H]` (Loads calculated 16-bit offset into SI).
- * `XCHG op1, op2` $\rightarrow$ `XCHG AL, AH` (Swaps AL and AH).

2. Arithmetic Instructions

Perform binary addition, subtraction, multiplication, and division.

- * `ADD dest, src` $\rightarrow$ `ADD AX, 0005H` ($AX = AX + 5$).
- * `SUB dest, src` $\rightarrow$ `SUB BX, CX` ($BX = BX - CX$).
- * `INC dest` $\rightarrow$ `INC SI` ($SI = SI + 1$).
- * `DEC dest` $\rightarrow$ `DEC DI` ($DI = DI - 1$).
- * `MUL src` $\rightarrow$ `MUL BL` (Unsigned multiply $AX = AL \times BL$).
- * `DIV src` $\rightarrow$ `DIV CL` (Unsigned divide $AL = AX / CL$, $AH = \text{remainder}$).

3. Logical Instructions

Perform bitwise logical operations and comparisons.

- * `AND dest, src` $\rightarrow$ `AND AL, 0FH` (Masks upper nibble).
- * `OR dest, src` $\rightarrow$ `OR BL, 80H` (Sets MSB to 1).
- * `XOR dest, src` $\rightarrow$ `XOR AX, AX` (Clears AX to 0).

- * `NOT dest` $\rightarrow$ `NOT CX` (Inverts all bits of CX).
 - * `CMP op1, op2` $\rightarrow$ `CMP AX, BX` (Subtracts BX from AX non-destructively to set flags).
-

4. Shift and Rotate Instructions

Bitwise shifting and rotation of operand contents.

- * `SHL dest, count` $\rightarrow$ `SHL AX, 1` (Logical left shift; multiplies by 2).
 - * `SHR dest, count` $\rightarrow$ `SHR BX, 1` (Logical right shift; unsigned divide by 2).
 - * `SAR dest, count` $\rightarrow$ `SAR CX, 1` (Arithmetic right shift; preserves sign bit).
 - * `ROL dest, count` $\rightarrow$ `ROL AL, 1` (Rotate left; MSB moves into LSB and CF).
 - * `ROR dest, count` $\rightarrow$ `ROR AL, 1` (Rotate right; LSB moves into MSB and CF).
-

5. Control Transfer Instructions

Alter instruction execution flow.

- * `JMP target` $\rightarrow$ `JMP START` (Unconditional jump).
 - * `JZ target` $\rightarrow$ `JZ EQUAL` (Conditional jump if Zero Flag ZF=1).
 - * `CALL target` $\rightarrow$ `CALL SUB_ROUTINE` (Saves IP to stack and jumps).
 - * `RET` $\rightarrow$ `RET` (Pops stack into IP to return from subroutine).
 - * `LOOP target` $\rightarrow$ `LOOP BACK` (Decrements CX; jumps if CX $\neq$ 0).
-

Q7. Explain all addressing modes of the 8086 microprocessor with syntax, effective address formula, and a worked numerical example for each mode.

Given Register & Memory State for Numerical Examples

Let: DS = 2000H, SS = 3000H, BX = 1000H, BP = 2000H, SI = 0100H, DI = 0200H, Displacement = 0050H.

1. Immediate Addressing Mode

- **Concept:** Operand is constant data given inside the instruction.
 - **Syntax:** `MOV reg, immediate_value`
 - **Example:** `MOV AX, 1234H` $\rightarrow$ `AX` gets 1234H. (No EA/PA calculation).
-

2. Register Addressing Mode

- **Concept:** Operand is inside a register.
 - **Syntax:** `MOV reg1, reg2`
 - **Example:** `MOV AX, BX` $\rightarrow$ `AX` gets contents of `BX` (1000H).
-

3. Direct Addressing Mode

- **Concept:** 16-bit offset is explicitly given in the instruction.
- **Syntax:** `MOV reg, [displacement]`
- **Formula:** `EA = displacement`, `Physical Address (PA) = (DS  $\times$  16) + EA`

- **Example:** `MOV AX, [0050H]`
 - `EA = 0050H`
 - `PA = (2000H × 10H) + 0050H = 20000H + 0050H = 20050H .`
-

4. Register Indirect Addressing Mode

- **Concept:** Offset is held in a base/index register (`BX` , `BP` , `SI` , `DI`).
 - **Syntax:** `MOV reg, [Base_or_Index_Reg]`
 - **Formula:** `EA = [Register]` , `PA = (Segment × 16) + EA`
 - **Example:** `MOV AX, [BX]`
 - `EA = BX = 1000H`
 - `PA = (DS × 16) + 1000H = 20000H + 1000H = 21000H .`
-

5. Based Addressing Mode

- **Concept:** EA is base register (`BX` or `BP`) plus displacement.
 - **Syntax:** `MOV reg, [Base_Reg + displacement]`
 - **Formula:** `EA = Base_Reg + displacement`
 - **Example:** `MOV AX, [BX + 0050H]`
 - `EA = 1000H + 0050H = 1050H`
 - `PA = (DS × 16) + 1050H = 20000H + 1050H = 21050H .`
-

6. Indexed Addressing Mode

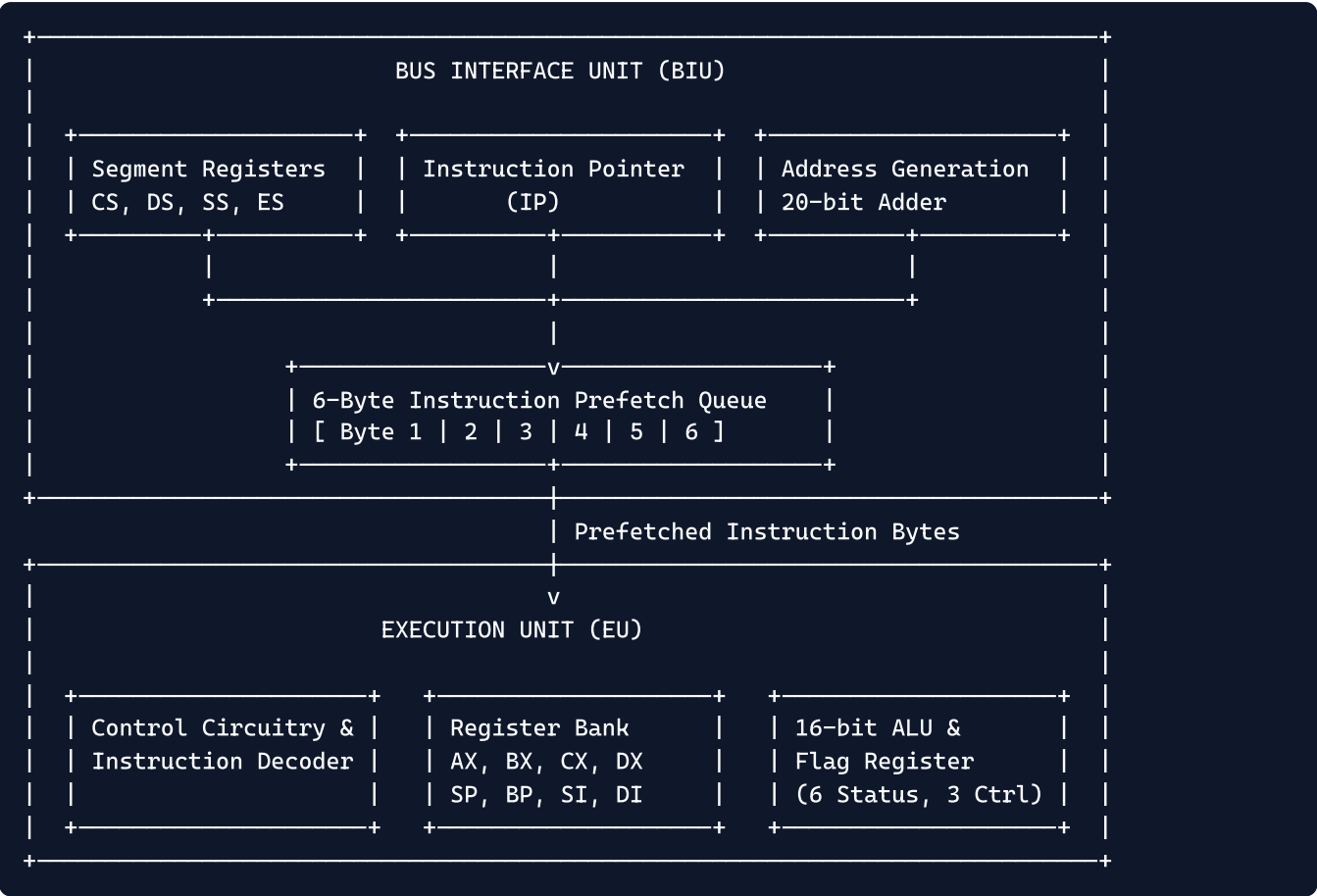
- **Concept:** EA is index register (`SI` or `DI`) plus displacement.
 - **Syntax:** `MOV reg, [Index_Reg + displacement]`
 - **Formula:** `EA = Index_Reg + displacement`
 - **Example:** `MOV AX, [SI + 0050H]`
 - `EA = 0100H + 0050H = 0150H`
 - `PA = (DS × 16) + 0150H = 20000H + 0150H = 20150H .`
-

7. Based-Indexed Addressing Mode

- **Concept:** EA is the sum of a base register, index register, and optional displacement.
 - **Syntax:** `MOV reg, [Base_Reg + Index_Reg + displacement]`
 - **Formula:** `EA = Base_Reg + Index_Reg + displacement`
 - **Example:** `MOV AX, [BX + SI + 0050H]`
 - `EA = 1000H + 0100H + 0050H = 1150H`
 - `PA = (2000H × 10H) + 1150H = 20000H + 1150H = 21150H .`
-

Q8. Draw and explain the block diagram of the 8086 microprocessor, labeling all registers, ALU, flag register, instruction queue, and explain how pipelining (BIU + EU) improves performance.

1. Detailed Block Diagram



2. Components Explanation

- **BIU:** Prefetches instructions, calculates 20-bit physical address, interfaces with memory bus.
- **EU:** Decodes and executes instructions, contains registers, ALU, and Flag register.
- **6-Byte Queue:** Stores prefetched instructions so the EU does not wait for memory fetches.
- **Flag Register (16-bit):**
 - *Status Flags:* CF (Carry), PF (Parity), AF (Auxiliary Carry), ZF (Zero), SF (Sign), OF (Overflow).
 - *Control Flags:* IF (Interrupt Enable), DF (Direction), TF (Trap/Single Step).

3. Pipelining in 8086 (BIU + EU Parallelism)

- **Non-Pipelined System:** CPU fetches instruction 1 $\rightarrow$ executes instruction 1 $\rightarrow$ fetches instruction 2 $\rightarrow$ executes instruction 2. Memory bus is idle during execution, CPU is idle during fetch.
- **8086 Pipelined System:**
 - While the **EU is executing** an instruction inside the processor, the **BIU independently fetches** subsequent instruction bytes from memory and stores them in the 6-byte prefetch queue.
 - Once the EU finishes execution, it immediately grabs the next instruction from the queue in 0 wait states without fetching from RAM.
 - **Performance Gain:** Overlaps fetch and execute phases, drastically increasing throughput and reducing total execution time.

Q9. Numerical: Given a program's instruction mix with respective clock cycles, calculate total CPU time, CPI, and MIPS.

Problem Statement

A 2.5 GHz ($2.5 \times 10^9 \text{ Hz}$) processor executes a program with a total of **40 million instructions** (40×10^6). The program consists of the following instruction mix:

1. **Arithmetic & Logical Instructions:** 45% of total instructions, requiring 2 clock cycles per instruction.
2. **Data Transfer Instructions:** 30% of total instructions, requiring 3 clock cycles per instruction.
3. **Control Branch Instructions:** 15% of total instructions, requiring 5 clock cycles per instruction.
4. **Memory Access Instructions:** 10% of total instructions, requiring 6 clock cycles per instruction.

Calculate:

1. Total number of clock cycles required by the program
2. Average CPI of the program
3. Total CPU Execution Time (in milliseconds)
4. Processor MIPS rating

Solution

Step 1: Calculate Instruction Counts per Category

- Total Instructions (IC_{total}) = 40×10^6
- $IC_{\text{arithmetic}} = 0.45 \times 40 \times 10^6 = 18 \times 10^6$ instructions
- $IC_{\text{data}} = 0.30 \times 40 \times 10^6 = 12 \times 10^6$ instructions
- $IC_{\text{branch}} = 0.15 \times 40 \times 10^6 = 6 \times 10^6$ instructions
- $IC_{\text{memory}} = 0.10 \times 40 \times 10^6 = 4 \times 10^6$ instructions

Step 2: Calculate Total Clock Cycles

$$\text{Total Cycles} = \sum (IC_i \times \text{CPI}_i)$$

$$\text{Total Cycles} = (18 \times 10^6 \times 2) + (12 \times 10^6 \times 3) + (6 \times 10^6 \times 5) + (4 \times 10^6 \times 6)$$

$$\text{Total Cycles} = (36 + 36 + 30 + 24) \times 10^6 = 126 \times 10^6 \text{ cycles}$$

Step 3: Calculate Average CPI

$$\text{Average CPI} = \frac{\text{Total Clock Cycles}}{\text{Total Instruction Count}}$$

$$\text{Average CPI} = \frac{126 \times 10^6}{40 \times 10^6} = 3.15 \text{ cycles per instruction}$$

Step 4: Calculate CPU Execution Time

$$\text{CPU Time } (T_{\text{cpu}}) = \frac{\text{Total Clock Cycles}}{\text{Clock Rate}}$$

$$T_{\text{cpu}} = \frac{126 \times 10^6}{2.5 \times 10^9} = \frac{126}{2500} = 0.0504 \text{ seconds} = 50.4 \text{ ms}$$

Step 5: Calculate MIPS Rating

$$\text{MIPS} = \frac{\text{Instruction Count}}{T_{\text{cpu}} \times 10^6}$$

$$\text{MIPS} = \frac{40 \times 10^6}{0.0504 \times 10^6} = \frac{40}{0.0504} = 793.65 \text{ MIPS}$$

Alternatively:

$$\text{MIPS} = \frac{\text{Clock Rate}}{\text{Average CPI} \times 10^6} = \frac{2.5 \times 10^9}{3.15 \times 10^6} = 793.65 \text{ MIPS}$$
